PA consistency machines: comparison and new source reductions

Research prepared for Vladimir Reshetnikov

13 November 2026

Abstract

This investigation did not obtain a two-symbol, single-tape blank-input machine with fewer than 196 working states and the requested PA-provable consistency biconditional. It compares the PA source snapshots and all twenty incoming archives at a pinned repository revision. The positive results are an explicit parameter-free MP command compiler adapted from the ZFC work; a 157-node source representation of the combined 3,573-instruction negation/MP program; and a source representation with the same count of 159 internal nodes and 3,812 instructions but no path containing six consecutive zeros. Independent finite control-graph checks verify the two representation changes. None is a smaller physical machine. New bounded local searches and rejected physical candidates are documented with reproducible artifacts. The report distinguishes source equivalence, physical refinement, and the uniform arithmetic proof translations needed for the requested endpoint.

1 Outcome and conventions

The intended endpoint is a concrete table M for which

$$\#\,\mathrm{WorkingStates}(M) \le 195, \qquad \mathrm{PA} \vdash \neg\,\mathrm{Halt}(M, \mathrm{blank}) \leftrightarrow \mathrm{Con}(\mathrm{PA}).$$

The alphabet is $\{0, 1\}$, the tape is two-way infinite and initially zero, and the separate halt control is excluded from the state count. These are the conventions of the supplied PA196 construction. Increasing the alphabet or changing the counting convention would not meet the intended comparison.

No table satisfying this endpoint is claimed in this packet. The reference tables are labelled as references, and rejected 195-state tables are labelled as rejected experiments. A smaller source expression, a favorable allocation estimate, a finite nonhalting prefix, and a proved consistency equivalence are different results.

The repository snapshot is

`38b84bce90b7d1c24c3319c0ff58d8ef92d96b21`.

The principal imported upstream revisions are James Chang's PA196 at `407cdc356f6b5866 35e71ac92be394255f4bfea4` and PA209 at `69d98241a721ead20b4f0b17df9062950b40fbfe`. The directory-level comparison, theorem inventory, and proof sources were read directly; archive inventories and selected tables were checked locally. The original repositories were not modified. A new full Lean build was not performed in this investigation.

2 What the existing machines establish

2.1 The internal theorem has a specific direction

Let H denote the exact arithmetic sentence expressing blank-tape halting of PA196. The imported theorem inventory and `Independence.lean` establish the internal implication

$$\mathrm{PA} \vdash \neg H \longrightarrow \mathrm{Con}(\mathrm{PA}).$$

The external Lean development also proves literal nonhalting in its metatheory. The imported repository audit explicitly leaves the converse internal implication as an obligation:

$$\mathrm{PA} \vdash H \longrightarrow \neg\,\mathrm{Con}(\mathrm{PA}), \quad \text{equivalently} \quad \mathrm{PA} \vdash \mathrm{Con}(\mathrm{PA}) \longrightarrow \neg H.$$

This is a statement about the supplied theorem statements and remaining proof bindings. It is not a claim that the converse is false or unprovable. Some incoming manuscripts give ordinary mathematical approaches to it; they do not export a new complete kernel-checked biconditional for the literal PA196 table. A sound ordinary proof could settle it, but its uniform trace and proof-code transformations must actually be supplied. See the primary sources [1, 2, 3].

The two missing directions can be isolated as

$$\text{literal halted trace} \longrightarrow \text{accepted native proof of code 1} \longrightarrow \text{PA refutation}.$$

The imported arithmetic transport gives the useful converse directions. An external satisfaction argument in a hereditarily finite model does not by itself translate arbitrary model-internal proof codes into PA proofs.

2.2 Main construction families

Family	States	Sites	Distinguishing feature
James Chang PA209	209	3,948	Direct predecessor of the imported PA196.
Historical U208	208	3,909	One-state context fold of its own native U209 parent.
James Chang PA196	196	3,812	Transposed quantifier code, changed axiom basis and reader sharing.
Combined negation/MP	197	3,573	Two source changes combined; a complete symbolic runtime receipt is supplied upstream.
Diagonal-vacuity variant	199	3,700	Shorter axiom sequence but more physical states.
Equivalent-gate variant	198	3,773	199 macro items; source and physical counts improve differently.
Integrated-witness variant	203	3,745	Best reported physical table for its shorter gate word.
H-only Shift variant	196	3,811	Different command arrangement and startup; same state count.

Table 1: Selected comparison points. Validation scopes differ; state count alone is not an assertion of the full requested consistency theorem.

PA196 has 36 engine states, 13 leaf-routing states, and 147 reader states. Its source contains 159 distinct internal subtrees. The paper's six AX reader identifications are already included in 196. They are not six additional savings that can be subtracted again.

The older U208 papers identify native roles 172 and 211 in their particular parent. Changing the survivor's label or helper changes a presentation, not the number of independent reductions. James Chang's PA209 is a distinct 209-state parent. In the PA209-to-PA196 change, both of those historical roles disappear with the replaced axiom body. A direct scan also finds no PA196 state having the exact mixed-entry motif needed for the original U208 transplantation. Other context-dependent constructions remain possible.

2.3 Twenty archives, with duplicate presentations separated

The complete archive inventory, including hashes and member counts, is retained in the packet. The following table describes the principal contribution of every incoming archive. Long names share the literal date suffix shown in the inventory.

Archive or archive family	Principal content and outcome
`U208_ConPA_2026-10-29.zip`	Original primary U208 and local fold argument.
`U208_ConPA_Conditional_Context_2026-10-29.zip`	Two original U208 layouts; conditional-context presentation.
`U208_ConPA_Entry_Context_2026-10-29.zip`	Entry-context presentation of the same two layouts.
`U208_ConPA_History_2026-10-29.zip`	History-based presentation of the same two layouts.
`u208_research_2026-10-29.zip`	Primary/Plain presentations of the two original classes.
`pa208_research_2026-10-29.zip`	Eighteen additional same-size helper variants.
`Below_U208_Research_2026-10-29.zip`	Local, offset and source-context searches; no accepted sub-208 table.
`ConPA_below208_research_2026-10-29.zip`	Source permutations, two-row searches and logical obstruction.
`pa_consistency_research_2026-10-29.zip`	Five-site source pruning; internal-node count increases.
`u208_comparison_research_2026-10-29.zip`	Concrete canonical-frame counterexample to a proposed 207-state decoder.
`pa_unary_power_cover.zip`	An alternate unary-cover theory and interpretation; no compiled table.
`conpa_tm_comparison_and_search.zip`	Accepted same-size A/D register swap; failed 195-state rotation and scanner shortcut.
`conpa_tm_comparison_and_reduction_search.zip`	Two explicitly experimental 196-state MP variants and command arguments.
`pa_tm_research_2026-11-13.zip`	A shorter-path 196-state layout and reverse-translation research.
`pa_tm_extended_research_2026-11-13.zip`	Four closing-decrement variants; stronger reachability and grammar searches.
`pa_consistency_research_2026-11-13.zip`	The 199-state diagonal-vacuity and 197-state negation constructions.
`pa_tm_comparison_and_reductions.zip`	Two rotated 196-state layouts and a 198-state equivalent-gate layout.
`pa_consistency_reduction.zip`	Rebracketing, H-only Shift, and integrated-witness gate experiments.
`pa_consistency_research.zip`	158-node sources, eight-zero scanner regrouping, logical counterinterpretations and head-order search.

Archive or archive family	Principal content and outcome
`pa196_research_2026-11-13.zip`	157-node source without a smaller table; refined local contexts and two arbitrary-row repair search.

A fresh replay distinguishes 27 designated final U208 presentations: the nine original presentations give two rooted action-preserving graph classes, while the eighteen added helper variants give eighteen further classes, for twenty classes in total. Calling the entire corpus "two U208 machines" would miss those later variants.

The separate 196-state census selects fourteen designated tables. Each has 196 graph-reachable working states, 392 binary transitions and one separate halt label; all fourteen are pairwise noniso-morphic as rooted, read-labelled transition graphs preserving write symbols and directions. Two are explicitly experimental MP variants. This census does not certify their arithmetic endpoints. The selection manifest preserves the exact member names, hashes, state maps, and original validation scopes.

3 A new 157-node representation of the combined program

The existing combined program incorporates the shorter negation macro and an MP success branch that returns its consequent in the cache. It has 3,573 primitive sites, 159 distinct internal source nodes and a supplied 197-state table. This investigation found a representation with

$$3,573 \text{ sites}, \qquad 157 \text{ internal nodes}, \qquad \text{maximum path length } 41, \qquad \text{maximum zero run } 6.$$

Its seed-site path remains `100000`.

The retained witness is checked independently of its search history. The verifier parses both complete binary trees, compares the ordered primitive instructions, and resolves every successor twice: by binary increment and path cutting, and by ancestor intervals. Both resolvers agree and every ordinal successor equals that of the supplied combined program.

Proposition 3.1 (Source representation invariance)**.** *Suppose two complete source trees have the same ordered primitive instructions and the same ordinal successor for every branch. Starting at the same ordinal site with the same register vector, their ordinal executions agree at every step, including halting.*

Proof. The increment and decrement operations have the same register argument; the zero/positive tests therefore agree. Their target ordinals agree by hypothesis. Break and continue have the same resolved successor and change no register. A halt is a halt in both. These observations give the one-step identity, and induction gives equality of every finite execution. This argument quantifies over arbitrary natural register contents. □

The proposition applies to the new witness and the selected combined program. It does not supply the separate logical compiler for the original MP modification, nor a physical interpreter for the new paths. A simple inherited-alias estimate for the new tree is 200 states because existing reader folds are lost. That figure is an allocation estimate, not a lower bound and not a verified table. The 157-node result is useful source compression, but it does not meet the requested 195-state endpoint.

4 A source with no six-zero path

The second exact witness preserves the original PA196 program itself: all 3,812 ordered primitive instructions and every ordinal control edge. It has 159 internal nodes, maximum path length 37, and maximum zero run 5. The original figures are 159, 38 and 6. The witness removes all twenty original paths containing six consecutive zeros without increasing the number of distinct internal nodes.

The seed ordinal 25 changes from `100000` to `10100000`. The source-preservation proposition proves equality of the abstract ordinal executions at that seed. It says nothing by itself about the binary machine's blank initialization, which must produce the corresponding encoded frame.

4.1 The coordinated outer reassociation

A useful first step is

$$((A, B), (C, D)) \quad \longmapsto \quad (A, ((B, C), D)),$$

where $A =$ `node7`, $B =$ `node20`, $C =$ `node170` and $D =$ `node23` in the pinned source. A, B, C have no escaping control depths. The final continue in D still has three right ancestors, so its target remains ordinal zero. The complete graph is unchanged even though checking the two component rotations in isolation is unnecessarily restrictive. The first path becomes `00000`; nineteen other six-zero paths initially remain.

For discovery, penalizing the *number* of six-zero paths supplies a useful search signal when merely penalizing the maximum run does not. The final verifier relies on the finite source identity and does not trust the heuristic or its random seed.

4.2 Why the scanner saving remains unallocated

The existing return scanner distinguishes the left marker from internal path bits by counting zeros. Shortening it by one state on the original program is already known to fail. Decrement-zero execution can introduce an additional temporary zero next to a long path run. A path representation with maximum zero run five is a concrete candidate for making the shorter scanner usable, but a complete machine must still be built and checked.

The naive outer-table rewire fails even before shortening the scanner. Native state 69 uses the preceding path bit to distinguish two source nodes. The reassociation moves one of its entries from a path ending in 1 to a path ending in 0, so it selects the wrong child. Our direct whole instruction checker rejects ordinal 11 on its zero-decrement branch; a search over every replacement action for native row 69/0 finds no one-row repair within the specified prefix-endpoint test. Simply deleting a scanner state does not repair this reader mismatch.

The original blank initialization visits the dispatcher 463 times before the advertised clean frame, beginning at step 38. A changed root cannot be assumed irrelevant to startup. The final five-zero witness therefore remains a source artifact, not a 195-state transition table.

5 A parameter-free MP command for the PA196 calculus

This construction transfers the opposite-comparison-residual idea from the repository's ZFC `k215-residual-parameter` experiment to PA196's different three-command language. It is a

source-level construction. No claim of a smaller physical machine or a newly kernel-checked PA theorem is made here.

Write $[a]W = P(a, W)$ for the exact unshifted Cantor stack constructor, $I(a, b) = 4P(a, b)$, and $U(i, a) = 4P(a, i) + 2$. Code 0 denotes truth and code 1 is the target contradiction. A store is (t, W, d, h). Retain Shift, which halts if $t + h = 1$ and otherwise maps

$$S(t, [a]W, d, h) = (h, W, d, a).$$

Replace the parameterized MP command by the nullary command

$$M(t, W, d, h) = \begin{cases} (d, [0]W, d, 0) & I(t, d) = h, \\ (0, W, d, 0) & I(t, d) \neq h. \end{cases}$$

For $A(c, x, y)$ put $D = d + x$, $H = h + y$. Emit exactly the old gate $g(H, D, c)$ and exactly the old eight heads $r_1, \ldots, r_8$, in their old top-to-bottom order; in particular $r_8 = U(d, h)$ and r_7 is the restricted implicational head with fourth parameter $d \leq D$. Change only the numeric residual:

$$A(c, x, y)(t, W, d, h) = \big(g, [r_1] \cdots [r_8][t]W, \max(c - D - 1, 0), 0\big).$$

Every emitted head and gate is different from 1, and $r_8 \geq 2$. These are literal constructor-tag facts, requiring no assumption that t, h, or the old stack are sound.

5.1 Exact command macros

Commands in a displayed word execute from left to right. Let $A_0 = A(0, 0, 0)$ and define

$$P = A_0 S^8 M, \qquad R = A_0 S^{10}, \qquad Z = (PR)^2, \qquad T_v = P\, A(v + 1, 0, v + 1) S^{10}.$$

The following equations hold for arbitrary natural stores, with no early halt:

$$\begin{aligned} P(t, W, d, h) &= (0, [t]W, 0, 0), \\ R(t, [a]W, d, h) &= (t, W, 0, a), \\ Z(t, W, d, h) &= (0, W, 0, 0), \\ T_v(t, W, d, h) &= (0, W, v, t). \end{aligned}$$

For P, A_0 sets the residual to zero. After eight Shifts the cache is $U(d, h)$, of residue 2 modulo 4; hence M fails regardless of the current value and returns zero without touching the saved old top. All tested adjacent fresh values differ from 1, so their sum cannot be 1. For R, the tenth Shift tests $U(d, h) + t \geq 2$, then exposes t and pops one old stack item. Applying PR twice proves the formula for Z. After P in T_v, both theorem fields and d are zero. The second AX has $D = 0$, $H = c = v + 1$, so its gate is zero and its residual is v. Its generalization is $U(0, 0) = 2$. Ten Shifts expose the stored original t in the cache and restore the original stack.

These equations use only decode$(P(a, W)) = (a, W)$. They therefore remain valid when $a = W = 0$ and $P(0, 0) = 0$; no injective distinction between an empty stack and a pushed zero is presumed.

5.2 A proof compiler

A compiler block for a derivable formula B is required, on any numerical store, either to halt or to return $(B, W, d', 0)$, preserving the old stack exactly. The parameter d' may be arbitrary. The following construction is structural induction on a derivation in PA196's exact numeric calculus.

Truth is compiled by Z. To select the jth AX head, $1 \le j \le 8$, start AX with current zero and follow it by

$$S^{j+1}R^{8-j}.$$

The initial Shifts place r_j in the current field. Subsequent Rs preserve that field and discard only the remaining fresh prefix, ending with the saved zero in the cache. For the restricted head with data (D, H, c, u), $u \le D$, the complete block is

$$Z\,T_u\,A(c, D-u, H)\,S^8R.$$

The other six heads use $u = 0$ and the appropriate selection index. For a permitted gate, use $ZA(c, D, H)R^9$. The nine pops remove the eight new heads and saved zero while retaining the gate as current.

If p_B compiles B, a block for $U(i, B)$ is

$$p_B\,T_i\,A_0S^9.$$

After T_i the store is $(0, W, i, B)$; the last eight-head item is exactly $U(i, B)$, and nine Shifts expose it with cache zero.

If p_I compiles $I(A, B)$ and p_A compiles A, modus ponens is compiled by

$$p_I\,P\,p_A\,T_B\,S\,M\,R.$$

Unless a premise block has already halted, the successive key stores are

$$(I(A,B), W, d, 0), \quad (0, [I(A,B)]W, 0, 0), \quad (A, [I(A,B)]W, d_1, 0), \quad (0, [I(A,B)]W, B, A).$$

The Shift halts when $A = 1$, which is acceptable. Otherwise it gives $(A, W, B, I(A, B))$, so M succeeds and gives $(B, [0]W, B, 0)$. The final R removes precisely that zero, yielding $(B, W, 0, 0)$. This also covers $I(0, 0) = 0$ literally. For a derivation of code 1, append one Shift to its compiler block: $(1, W, d', 0)$ passes the exact halt test.

Soundness uses the usual invariant: the current value, cached value, and every numeric stack entry are derivable. Shift can halt only when one of the two theorem values is 1 and the other is 0. Successful MP is literal modus ponens, failed MP returns truth, and the altered AX emits the same formulas as before. Its numeric residual has no logical invariant. Thus the revised command calculus searches for exactly the same refutations.

5.3 Changed arities and fair enumeration

Let $e(\epsilon) = 0$ and encode a nonempty continuation by

$$\begin{aligned} e(Sw) &= 1 + P(0, e(w)), \\ e(Mw) &= 1 + P(1, e(w)), \\ e(A(c, x, y)w) &= 1 + P(c+2, P(y, P(x, e(w)))). \end{aligned}$$

These are the old tags; only MP no longer consumes an additional operand field. The untouched scheduler subtracts one from positive continuations and decodes the outer pair. Deleting its old 14-site reader implements exactly this new convention. Each positive continuation decreases under decoding, so a finite word is consumed completely. This is a claim about the rewritten source, not an assertion that old certificate integers have their old meanings.

The distinction between a continuation and a fresh candidate is essential. For nonempty w, put $f(w) = e(w) - 1$. If the continuation register is $L = e(w) > 0$, the scheduler decodes $L - 1 = f(w)$ and leaves its fresh candidate counter N unchanged. If $L = 0$ and $N = f(w)$, it decodes N without subtracting one and increments that counter. Both cases therefore decode the same first command and leave $L = e(w')$ after its parameter fields, where w' is the tail. This is a structural induction on w, using the exact Cantor inverse identities. The original source supplies precisely these two interfaces as `Fetch.positive` and `Fetch.fresh` in `PA196/Source/Fetch.lean`; its `FetchDispatch.seed` in `Source/FetchDispatch.lean` is literally N when $L = 0$, and $L - 1$ otherwise. The source splice changes neither of those routines. Parameter-free MP consumes no field after its tag, whereas AX still consumes y and then x.

The compiler contract is uniform in the incoming numerical store, hence cofinality is obtained by prefixing a fixed compiled refutation with $A(Y, 0, 0)Z$. If z is the code of the fixed suffix, the fresh-candidate integer is $P(Y + 2, P(0, P(0, z)))$. It is strictly increasing and unbounded in Y, so an increasing candidate enumeration beginning at 2 includes such a halting word. Conversely, every halt from a theorem-safe seed is a derivation of 1. The needed coding and block arguments use finite induction and primitive-recursive proof transformations; a PA-internal binding to a literal TM still requires the separate machine formalization.

5.4 Exact source splice and its present cost

In the pinned PA196 source, replace the cleanup subtree $(\mathrm{CD6}, \mathrm{CC2})$ at old sites 3687–3688 by $(\mathrm{CD6}, (\mathrm{CI4}, \mathrm{CC2}))$ within the second comparison $[3680, 3692)$. The paired comparison loop consumes one excess scratch unit on inequality. Its remaining scratch value is $\max(c - D - 1, 0)$, transferred to initially zero register 4; registers $5, 6$ end zero. Then remove the closed MP reader $[3693, 3707)$. Every unchanged instruction and both of its ordinal successors are checked under the explicit splice map by `residual_source.py`. Two independent CFG resolvers agree. The isolated comparison was also replayed for all $0 \le c, D < 64$; this finite replay supplements the loop argument and does not replace it.

Here is the complete arbitrary-natural comparison argument. At its entry the registers (T, H, X, d) are $(t, D, c, 0)$. Number the thirteen new instructions $0, \ldots, 12$ and call the exit 13. Their resolved effects are

0	dec $H : (4, 1)$	1	dec $X : (3, 2)$
2	jump 0	3	jump 5
4	dec $X : (11, 5)$	5	dec $H : (7, 6)$
6	jump 5	7	dec $X : (10, 8)$
8	inc d; 9	9	jump 7
10	jump 13	11	dec $T : (13, 12)$
12	jump 11		

where the two targets of a decrement are zero and positive, respectively. After $k \le \min(D, c)$ complete trips through $0, 1, 2$, the invariant is $(T, H, X, d) = (t, D - k, c - k, 0)$ and the control is 0. Each trip takes three steps. The terminal cases are:

- If $D = c$, rows $0, 4$ enter 11 with $H = X = 0$. The $11, 12$ loop takes $2t + 1$ steps and clears T; d remains zero.
- If $D < c$, rows $0, 4$ enter 5 with $(H, X, d) = (0, c - D - 1, 0)$, leaving T untouched.
- If $D > c$, rows $0, 1, 3$ enter 5 with $(H, X, d) = (D - c - 1, 0, 0)$, again leaving T untouched.

Rows $5, 6$ clear H in $2H + 1$ steps. In rows $7, 8, 9$, if the incoming X is r, after $j \le r$ trips the invariant is $(X, d) = (r - j, j)$; each trip takes three steps. The zero test and exit jump take two more steps. Thus the exact comparison duration is

$$\begin{cases} 3D + 2t + 3, & D = c, \\ 3c + 2, & D < c, \\ 2D + c + 4, & D > c. \end{cases}$$

The final store is always $(T, H, X, d) = (\text{if } D = c \text{ then } 0 \text{ else } t, 0, 0, \max(c - D - 1, 0))$. Every other register is unchanged. This proves termination and the residual contract by elementary natural-number induction, including both zero boundary cases. The preceding unchanged guard copies c twice, performs the first $H = c$ test, then moves old D to H, leaving the residual destination $d = 0$ exactly as required. Consequently the final gate tests are still the two original equalities; only the disposable excess X is now retained as a numeric parameter.

The residual transfer has *exactly the same register-machine execution time* as the old comparison, for every t, D, c. In the old cleanup, one positive X cycle consists of decrementing X, jumping back to the H test, and testing the already zero H before returning to the X test: three steps. In the new cleanup, the corresponding cycle decrements X, increments d, and jumps directly back to the X test: also three steps. The initial H drain and both exit tests are unchanged. The shared prefix, the equality branch, and the $D > c$ branch have unchanged step counts as well. Applying the two cycle invariants to arbitrary r proves that both comparisons obey the three duration formulas above, not just that a finite selection of runs agrees. The constructor additionally checks both old and new outputs and equal durations in all 4,096 probes. Thus the numeric information was available in an otherwise wasted loop test. This does not imply equal physical TM time: the new source address layout still has to be compiled and verified.

The result has $3,799$ source instructions and $4,941$ guarded cases. It has 164 distinct internal subtrees versus 159 in PA196 because the changed comparison loses existing sharing. The source SHA-256 is `9aa46652ee8f11d7919700f860578d3c4c6b0ad8ec43524af79ec7d4703dff4a`. A conservative startup-preserving allocator with no new labels reports an empty domain. This is a limitation of that allocation model, not an impossibility theorem. No physical state reduction is established.

6 New physical search and concrete rejections

6.1 Extension of the two-row near misses

The incoming refined-context search admits 2,219 control/window pairs and exhausts its stated two-arbitrary-row repair template. Its parent/child endpoint relation preserves the entire five-cell tape window, head position, and projected control after a positive block of at most sixteen child steps. It considers one identified pair of working states and at most two changed remaining transition rows.

We reran that search with an additional targeted third-row stage. The first two stages reproduce 38,220 ordered state pairs, 118,504,736 first-row actions, 573,987,568 second-row actions, and 633 representative-context passes. For each of these 633 near misses, intersect the sets of rows visited by its failing complete obligations. A third repair must touch a row in that intersection. Exclude the first two changed rows and the removed state; then enumerate every legal action at every remaining eligible row.

This tests 1,400 eligible third-row keys and 1,097,600 third-row actions. There are no survivors. The third helper's own contexts are checked as well, so a local repair cannot silently break the helper's original role.

Scope: this is exhaustive for extensions of the selected near misses, not for all possible three-row repairs. The earlier pruning was designed for two changed rows and can exclude candidates that require three changes before even a representative test passes. The result is not a lower bound on 195-state machines.

6.2 Fourteen distinct 196-state variants

For each designated table we separately recompute the least radius-six blank-start window closure, project it to five cells, and run the same two-arbitrary-row search plus targeted third-row extension. The invariant begins with the all-zero window in state zero, follows every transition, and allows both values for a newly exposed cell. Induction on literal time therefore covers every actual blank-start window, while allowing spurious ones. Each table has its own invariant; the original PA196 contexts are not reused for a different machine.

Variant	Contexts	Second-row actions	Third-row actions
Closing CD3	2,782	406,092,400	406,112
Closing CD4	2,782	408,226,448	700,896
Closing CD5	2,782	404,192,768	700,896
Closing CD6	2,782	413,685,440	700,896
Eight-zero regrouping	2,807	458,745,840	634,256
H-only Shift	2,762	418,796,336	700,896
MP, no zero push (experimental)	2,774	416,468,640	700,896
MP, result in H (experimental)	2,781	416,717,168	700,896
Published PA196	2,774	417,538,016	700,896
Rebracketed	2,768	428,383,088	868,672
A/D register swap	2,774	417,538,016	700,896
Rotated	2,776	420,332,192	700,896
Rotated macro62	2,768	423,098,144	700,896
Short-path alternative	2,756	424,496,800	700,896

Table 3: New searches with separately computed radius-six invariants. Every row has zero surviving two-row or targeted three-row repairs.

Across these fourteen inputs the search considers 535,080 ordered state pairs, 1,657,293,680 first-row actions, 5,874,311,296 second-row actions and 9,618,896 third-row actions. The third stage extends 5,097 representative-context near misses. No survivor is reported. The larger counts do not strengthen the simulation template beyond its stated scope. In particular, all fourteen radius-six closures admit an abstract halt; they are safe overapproximations, not nonhalting proofs.

A synthetic four-state positive control uses exactly the same C++ search logic and finds seventeen reported reductions: one one-row, five two-row, and eleven three-row repairs. A separately written Python simulator checks all 2,176 control/window obligations for these outputs, including every original state and all thirty-two five-cell windows. This provides a nonvacuity check of the implementation, not a completeness proof of the search.

Failed searches under these overapproximations do not exclude a successful proof using a sharper reachable invariant, a wider window, longer blocks, multiple parent steps, several simultaneous state

identifications, changed tape encodings, or a different source algorithm.

7 The remaining arithmetic bridge

7.1 A useful general reverse-refinement lemma

Lemma 7.1 (Positive total segments permit bounded reversal)**.** *Let a deterministic source program have a primitive-recursive totalized step function, with a distinguished halting instruction, and let the physical machine also be deterministic. Suppose each nonhalting source step has a total positive-length physical simulation ending at its next encoded working frame, without a physical halted configuration anywhere in that segment, including its endpoints. Suppose the source halting instruction has a finite segment ending in a physical halt. Suppose these contracts and a finite nonhalting startup ending at the initial encoded working frame are proved inside PA. Then PA proves equivalence of source and physical halting.*

Proof. The forward direction concatenates the finitely many source-step segments before the source halt and then the halting segment. For the reverse direction, let a physical halt occur within t steps after startup. Assume that the first $t+1$ source instructions are all nonhalting. Concatenating their positive nonhalting segments gives a physical nonhalting prefix of length at least $t+1$. Determinism identifies its first t transitions with the proposed halted trace, a contradiction. Thus a source halt occurs among those $t+1$ instructions. The proof uses PA induction up to the given trace length and coded finite concatenation, not an inverse control-state map. With absorbing totalized halt states, a segment ending at a nonhalt frame likewise cannot have passed through halt. □

The lemma makes the missing hypotheses precise. In particular, a disjunction allowing an early physical halt or a next source frame is weaker than the nonhalting-segment contract used here. Merely knowing that a source halt can be simulated is insufficient for the reverse implication.

7.2 Syntactic soundness rather than a uniform truth predicate

For native logical soundness in PA, the promising direct interpretation is

$$M(a,b) \mapsto \mathrm{Bit}(x_a, x_b), \qquad E(a,b) \mapsto x_a = x_b,$$

with transposed quantifiers translated directly. PA proves $\mathrm{Bit}(u,v) \to v < u$, so the target code 1 translates to a contradiction. Code 0 is a truth alias and needs an explicit base case.

For each input formula $\varphi(z)$, PA's induction schema gives a bounded bit-comprehension proof: there is a number whose first n bits agree with φ on $z < n$. A syntactic proof-producing template can substitute the input formula into that fixed induction construction. This avoids asserting a PA truth predicate for arbitrary formulas. It must handle capture-free substitution, freshness, equality of variable labels, and the unshifted Cantor stack convention $P(0,0) = 0$.

The supplied repository discussion already develops these ideas, including the different equal-label gate case and a certificate-stack invariant with ghost entries beyond a zero numeric tail. This investigation does not claim a new implemented and internally verified proof-code compiler for the full bridge. The new residual command compiler leaves the native calculus unchanged; it does not eliminate that pre-existing obligation.

8 What transfers from ZFC and concise iPRA

The residual-parameter construction above is a concrete transfer from the ZFC research, adapted to PA196's different three-command interface. Other reusable ideas are contextual reader sharing, closed-source factoring, explicit macro contracts, and positive-segment refinement. The ZFC consistency target itself must be replaced by a PA-calibrated native calculus; a small ZFC machine is not automatically a Con(PA) machine.

The concise-iPRA program optimizes the size of a straightforward proof checker. The present problem counts binary tape states, including the interpreter, routing, and initialization. A smaller checker description can increase the physical state count after compilation. Also, simply replacing the PA checker with an iPRA checker changes the consistency statement. A proposed intuitionistic route of HA strength would need its PA calibration and exact compiler to be exhibited before its state budget could be compared. No such replacement is claimed here.

9 Reproduction and artifact status

The ZIP contains this TeX source and its compiled PDF, the explicit new source witnesses, portable verifiers, exact reference-table census, provenance and licenses, and the new search source and receipts. The top-level `README.txt` gives the precise quick and extended commands. Quick checks require Python 3; rerunning the local synthesis also requires a C++17 compiler. The artifacts do not depend on a signed-in GitHub session.

Artifact	What its check establishes
Combined 157-node witness	Exact ordered primitive word and two independently resolved control graphs equal those of the combined 197-state source.
159-node/five-zero witness	Exact original PA196 source graph; complete path geometry and node count.
Residual source and compiler	Explicit command algebra, changed-arity encoding, exact retained-source splice checks, finite regressions with stated models.
196-table census	Literal table shape, graph reachability, exact canonical graph comparison and source provenance.
Local search receipts	The specified bounded search was performed; no general machine-size lower bound.
Rejected scanner experiment	The root mutation violates the intended decoder-prefix specification and is rejected by the instruction-contract checker.

10 Further research questions

1. Can the exact five-zero source be allocated with a six-zero return scanner while preserving enough contextual reader sharing to produce 195 states? The seed and complete blank initialization must be rebuilt.

2. Can the residual transfer be factored into an already needed loop, so that deleting MP's

parameter reader saves physical reader roles rather than duplicating the comparator? Its command compiler is now explicit.

3. Can a physical allocator optimize helper use and source-tree structure jointly, with full runtime and startup constraints, instead of pricing internal nodes and repairing lost aliases afterward?

4. Can a multi-instruction simulation merge states that no single-parent-step local test can merge? A useful certificate format would record resynchronization frames and a positive progress bound.

5. Can the native-to-PA syntactic proof compiler and the bounded reverse trace argument be implemented against the exact PA196 arithmetic predicates, including nonstandard proof codes and zero-alias stacks?

References

[1] V. Reshetnikov, ProveIt repository, *PA196 and the incoming U208 construction*, `Computability/BusyBeaver/Research/PA/pa196-vs-u208/COMPARISON.md`, revision `38b84bce90b7`. `https://github.com/VladimirReshetnikov/ProveIt/blob/38b84bce90b7d1c24c3319c0ff58d8ef92d96b21/Computability/BusyBeaver/Research/PA/pa196-vs-u208/COMPARISON.md`.

[2] ProveIt, *PA196: a route to the missing internal implication*, `PA-BRIDGE.md`, same revision. `https://github.com/VladimirReshetnikov/ProveIt/blob/38b84bce90b7d1c24c3319c0ff58d8ef92d96b21/Computability/BusyBeaver/Research/PA/pa196-vs-u208/PA-BRIDGE.md`.

[3] J. Chang, *Nonhalting and PA-independence for a 196-state Turing machine*, imported paper, theorem inventory, and Lean sources. `https://github.com/VladimirReshetnikov/ProveIt/tree/38b84bce90b7d1c24c3319c0ff58d8ef92d96b21/Computability/BusyBeaver/Research/PA/jamesyc-upstream/pa196`.

[4] J. Chang, PA209 snapshot, revision `69d98241a721ead20b4f0b17df9062950b40fbfe`. `https://github.com/jamesyc/BusyBeaver/tree/69d98241a721ead20b4f0b17df9062950b40fbfe`.

[5] ProveIt, twenty incoming research archives, exact names and input digests in the accompanying inventory. `https://github.com/VladimirReshetnikov/ProveIt/tree/38b84bce90b7d1c24c3319c0ff58d8ef92d96b21/docs/incoming`.

[6] ProveIt, combined negation/MP experiment and local invariant audit. `https://github.com/VladimirReshetnikov/ProveIt/tree/38b84bce90b7d1c24c3319c0ff58d8ef92d96b21/Computability/BusyBeaver/Research/PA/pa196-vs-u208/source-search/negation-mp197`.

[7] ProveIt, *Parameter-free MP on the accepted215 source*, ZFC residual-parameter experiment and ordinary proof. `https://github.com/VladimirReshetnikov/ProveIt/tree/38b84bce90b7d1c24c3319c0ff58d8ef92d96b21/Computability/BusyBeaver/Research/ZFC/k215-residual-parameter`.

[8] ProveIt, concise iPRA research. `https://github.com/VladimirReshetnikov/ProveIt/tree/38b84bce90b7d1c24c3319c0ff58d8ef92d96b21/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`.